

Next-Generation SIEM and SOAR Architectures: Integrating Deep Learning for Automated Alert Validation and Triage

The operational paralysis currently afflicting modern Security Operations Centers (SOCs) is driven by a fundamental misalignment between the volume of automated security alerts and the human capacity required to validate them. Automated detection logic—whether deployed at the endpoint, the network boundary, or the identity layer—is intentionally calibrated for high sensitivity. Security vendors operate under the mandate that generating extraneous noise is preferable to missing a catastrophic breach. Consequently, the burden of proof is shifted entirely onto Tier 1 (Triage) and Tier 2 (Incident Response) analysts, who must engage in exhaustive manual verification to differentiate true threats from false positives.

To resolve this crisis, the architecture of the Security Information and Event Management (SIEM) system must evolve from a passive aggregation engine into an active, machine-learning-driven inference platform. By embedding Deep Learning (DL) models directly into the SIEM layer and orchestrating the operational pivot-work through Security Orchestration, Automation, and Response (SOAR) platforms, the manual verification bottleneck can be systematically eliminated. This report provides an exhaustive architectural blueprint for developing and implementing an AI-native SIEM/SOAR environment. It evaluates the most robust open-source datasets available for cybersecurity machine learning, identifies the optimal neural network architectures for automated alert validation, and outlines the precise integration mechanics required to operationalize these models using the Wazuh SIEM and Shuffle SOAR platforms.

The Taxonomy of the Manual Verification Bottleneck

The crisis shared between Tier 1 and Tier 2 analysts can be deconstructed into specific operational failures that current deterministic security tools cannot resolve. Understanding these failure points is a prerequisite for designing targeted machine learning interventions. The first failure point is the False Positive Validation Bottleneck. Automated systems frequently flag standard administrative actions, such as PowerShell backup scripts or scheduled tasks running at off-hours, as malicious behavior based on superficial heuristics. To disprove the alert, an analyst must manually trace the execution history, verify the parent process, and analyze the behavioral baseline of the user. Traditional rule tuning cannot solve this, as reducing sensitivity invariably creates blind spots for attackers to exploit. The resulting noise represents a permanent tax on human labor.

The second failure point is Context Stitching. Modern enterprise environments suffer from profound tool sprawl, with security tools acting as isolated silos. An alert indicating a suspicious

outbound connection cannot be trusted in isolation; it requires the analyst to manually copy data across multiple platforms. The analyst must check firewall throughput, open the Endpoint Detection and Response (EDR) platform to trace the local process that initiated the connection, and check Active Directory to see who was authenticated at the time. The lack of a unified ontological framework means human analysts act as the inefficient integration layer between these isolated systems.

The third failure point involves Manual Data Enrichment and Intel Verification. When a system connects to an unknown IP address, automated platforms often rely on static, rapidly decaying threat intelligence feeds. Differentiating between a newly hijacked, legitimate content delivery network and a dedicated command-and-control server requires human intuition and extensive manual querying against external databases like VirusTotal, Shodan, or Cisco Talos. Analysts cannot block enterprise connections based on hunches, making manual threat intelligence validation mandatory.

The final and most technically demanding failure point is Code and Script Deobfuscation. Attackers continuously utilize "living off the land" binaries (LOLBins) and heavily obfuscated scripting languages to execute fileless malware. An analyst presented with a scrambled, Base64-encoded command line containing randomized variables and concatenated strings must manually decode the payload through multiple layers of obfuscation using open-source sandboxes or decoders like CyberChef. Automated signature-based tools flag the presence of obfuscation, but they consistently fail to safely interpret the underlying semantics.

To automate this entire validation workflow, a multi-modal machine learning approach is required. The system must natively parse process relationships as mathematical graphs, interpret obfuscated code as structural language, and evaluate sequential log events as temporal series.

Data Foundations: Open-Source Datasets for Security Machine Learning

The efficacy of any deep learning model is entirely bounded by the quality, volume, and relevance of its training data. For a custom ML-driven SIEM, training on generic data is insufficient; the datasets must directly reflect the complexities of enterprise telemetry, including obfuscated scripting, advanced persistent threat (APT) process graphs, and massive-scale system logs. The development of an internal model necessitates leveraging the most rigorous open-source datasets available in the cybersecurity domain.

Dataset Name	Primary Application Domain	Structural Format	Key Characteristics and Volume	Source / Affiliation
PowerShellCorpus	Script Deobfuscation	Text / Abstract Syntax Tree	408,665 scripts; labeled subset for	Revoke-Obfuscation ¹

			obfuscation; robust variety of offensive frameworks.	
DARPA OpTC	Provenance Graph Analysis	CDM Graph (V, E)	17 billion host/network events; 1,000 endpoint hosts; multi-stage APT engagements.	DARPA Transparent Computing ³
DARPA TC E3	Provenance Graph Analysis	CDM Graph (V, E)	High-fidelity causal event tracking; specific advanced attack sequences and background noise.	DARPA Transparent Computing ⁶
BGL	Log Anomaly Detection	Sequential Text Logs	Blue Gene/L supercompute r logs; line-by-line manual expert labels; 4.7 million messages.	Loghub / HPC Systems ⁹
HDFS	Log Anomaly Detection	Sequential Text Logs	Hadoop Distributed File System logs; block-level labels; 11 million messages.	Loghub / Distributed Systems ¹²

The PowerShellCorpus and Abstract Syntax Tree Corpora

PowerShell remains a primary vector for post-exploitation activities and fileless malware. Attackers exploit its tight integration with the .NET framework to execute scripts directly from memory, deliberately bypassing disk-based antivirus signatures. To train a model capable of automating the deobfuscation and intent-verification steps, the training corpus must contain a vast array of both benign and malicious scripts, spanning various obfuscation techniques. The most authoritative open-source dataset for this purpose is the PowerShellCorpus curated by researchers Daniel Bohannon and Lee Holmes². This corpus was explicitly developed to train the Revoke-Obfuscation framework and consists of approximately 408,665 unique PowerShell scripts harvested from public repositories, including GitHub, the PowerShell Gallery, and the TechNet Script Center¹. The corpus represents an uncompressed volume of roughly 4GB and encompasses scripts authored by over 28,000 individuals¹⁶. It incorporates outputs from prominent offensive obfuscation frameworks, including Invoke-Obfuscation, Invoke-CradleCrafter, and ISEsteroids, providing realistic representations of adversarial techniques such as string concatenation, tick-mark insertion, and format operator abuse¹. Crucially, a subset of this corpus has been meticulously labeled into binary classifications of "Obfuscated" and "Not Obfuscated"¹⁶. The labeling process specifically prioritized the operational question of whether a script would warrant human review during an incident response investigation, rather than simply labeling it "malicious" or "benign"¹⁶. This is an important distinction, as many legitimate IT administration scripts exhibit complex logic, while many malicious commands are entirely unobfuscated. The labeled subset includes 7,000 scripts from human authors (1,600 of which are obfuscated) and an additional 4,000 scripts that were intentionally scrambled¹⁶. By transforming these scripts into Abstract Syntax Trees (ASTs) to extract thousands of structural features, this dataset serves as the premier foundation for training structural natural language processing models to classify script intent².

DARPA Transparent Computing and Provenance Graphs

To automate the context stitching and process tree validation tasks, models must be trained on data that accurately represents the causal relationships between system entities over time. Traditional tabular datasets (such as the decade-old KDD datasets) fail to capture the structural behavior of modern attacks, which often manifest as long-range causal chains spanning multiple processes, files, and network connections⁵. This telemetry is best represented mathematically as a provenance graph.

The Defense Advanced Research Projects Agency (DARPA) Transparent Computing (TC) Program datasets, particularly the Operationally Transparent Cyber (OpTC) and E3 datasets, provide the most comprehensive foundation for graph-based intrusion detection³. The OpTC dataset encompasses over 17 billion events collected from an enterprise network comprising 1,000 Windows 10 hosts³. It captures a wide spectrum of both benign baseline activities and orchestrated advanced persistent threat (APT) red-team engagements¹⁸.

The DARPA TC datasets utilize a highly structured Common Data Model (CDM), which normalizes heterogeneous host and network telemetry into a unified semantic format³. The

CDM defines core entities—including Hosts, Principals, Subjects, and Objects—and the granular information flow dependencies between them, capturing UUIDs, timestamps, and predicate object paths³. This allows for the construction of explicit provenance graphs defined as $G = (V, E)$, where V represents the system entities (nodes) and E represents the temporally ordered interactions (edges)³. The datasets include detailed red-team attack chains featuring PowerShell Empire staging, lateral movement, privilege escalation, and data exfiltration³. Despite challenges with severe class imbalance (the vast majority of the 17 billion events are benign), these datasets provide the exact ground-truth required to train deep Graph Neural Networks capable of autonomously navigating process trees³.

Sequential Log Datasets: HDFS and BGL

For alerts originating from traditional firewalls, network appliances, and identity management systems, the analytical task is to identify deviations within highly structured, sequential log streams. The HDFS (Hadoop Distributed File System) and BGL (Blue Gene/L) datasets are the established benchmarks for evaluating deep learning-based log anomaly detection models⁹. The HDFS dataset contains approximately 11 million log messages generated by a cluster of over 200 Amazon EC2 nodes, with anomalies labeled at the block execution level⁹. The BGL dataset contains roughly 4.7 million log messages generated by a Blue Gene/L supercomputer, with independent labels assigned to each log line by domain experts¹². These datasets are critical for training models to understand the normal cadence and sequential probability of system events, thereby allowing the SIEM to automatically suppress alerts generated by benign but rare administrative operations, which are a primary source of Tier 1 false positives¹⁰.

Deep Learning Model Architectures for Alert Validation

No single machine learning model can independently resolve the diverse challenges of the manual verification workflow. The optimal approach requires an ensemble architecture, where specialized deep learning models are deployed against specific data modalities. The outputs of these localized models are then synthesized to produce a final, highly confident verdict on whether an alert is a true positive or a benign anomaly.

Analytical Domain	Target Data Modality	Recommended Architecture	Implementation Frameworks	Primary Objective
Context Stitching	Provenance Graphs (V, E)	Spatio-Temporal Graph Neural Networks (GNN)	Torch Geometric, GraphSAGE	Process tree analysis, attack path reconstruction.

Script Deobfuscation	Executable Code / Scripts	AST-based Transformers (CodeBERT)	HuggingFace, Tree-sitter	Semantic intent verification, obfuscation bypass.
Log Anomaly Triage	Sequential Log Text	LLM Classifiers / LSTM Autoencoders	vLLM, llama.cpp, PyTorch	Alert severity classification, sequence validation.

Graph Neural Networks for Provenance and Causal Analysis

The primary bottleneck for analysts is the manual reconstruction of process trees within EDR dashboards to establish intent. Traditional machine learning models process flat, tabular data, effectively stripping away the relational context that is essential for tracing an attack path²⁴. Furthermore, attackers easily evade flat-feature detection by subtly altering their behavior. To automate this step, the SIEM must utilize a Graph Neural Network (GNN).

GNNs operate directly on graph-structured data, making them inherently suited for provenance graphs²⁴. Advanced architectures, such as Spatio-Temporal Graph Neural Networks (ST-GNNs) or GraphSAGE, utilize a permutation-invariant message-passing mechanism to generate rich representations (embeddings) of each node²⁷. During each iteration, a node aggregates feature information from its local neighborhood, allowing the network to capture complex, multi-hop causal dependencies that human analysts struggle to manually trace²⁸.

The mathematical formulation for a typical message-passing layer in a GNN can be expressed as:

$$h_v^{(k)} = \sigma \left(W^{(k)} \cdot \text{CONCAT} \left(h_v^{(k-1)}, \text{AGGREGATE}^{(k)} \left(\left\{ h_u^{(k-1)} \mid \forall u \in \mathcal{N}(v) \right\} \right) \right) \right)$$

Where $h_v^{(k)}$ is the feature vector of node v at layer k , $\mathcal{N}(v)$ represents the set of

neighboring nodes, and σ denotes a non-linear activation function⁸.

When evaluating a new EDR alert, the SIEM constructs a local provenance graph surrounding the implicated process. The GNN analyzes this subgraph and compares its topological structure against known malicious execution patterns learned from the DARPA datasets²⁶.

Architectures such as StageFinder leverage an LSTM-based temporal estimator on top of the GNN embeddings to not only classify the subgraph as anomalous but to map the behavior

directly to specific MITRE ATT&CK stages, such as Lateral Movement or Privilege Escalation³¹. By employing node-masking and edge-reconstruction techniques within a Graph Attention Autoencoder, the model can identify the exact nodes contributing to the anomaly⁸. This provides the human analyst with a highly interpretable, mathematically verified attack path, entirely eliminating the need for manual process tree reconstruction.

Abstract Syntax Tree Transformers for Script Deobfuscation

To resolve the bottleneck of manual script deobfuscation, NLP techniques must be adapted to understand programming languages. Standard NLP models that process raw script text frequently fail because attackers can easily bypass text-based signatures using string concatenation, variable scrambling, or arbitrary whitespace insertion¹⁵. For example, a signature looking for the string `http` can be trivially bypassed by an attacker writing `'h' + 'ttp'` or utilizing PowerShell's `-f` format operator¹⁵.

The superior approach is to parse the raw script into an Abstract Syntax Tree (AST) before feeding it into the deep learning model. An AST strips away trivial syntactic variations and represents the hierarchical semantic structure of the code¹⁸. Regardless of how heavily a PowerShell command is obfuscated at the string level, its underlying AST will still reliably exhibit the structure of an Invoke-Expression node linked to a network download command³⁶. Tools like Tree-sitter apply context-free grammars to convert plain text into these structured node hierarchies³⁴.

The optimal model for this task is a fine-tuned pre-trained Transformer, such as CodeBERT, combined with an AST feature extraction pipeline. As demonstrated by frameworks like PASTRAL, absolute paths from the root to each token in the AST can be extracted to capture structural relationships³⁵. These syntactic paths are converted into numeric embeddings and combined with semantic language model embeddings³⁵. Alternatively, robust Random Forest classifiers trained on hybrid features—including AST node depth, function distribution counts, and textual tokens—have demonstrated classification accuracies exceeding 98% against heavily obfuscated PowerShell scripts¹⁸. By automating AST generation and passing the structural embeddings through a classification model, the SIEM can instantly determine whether an unreadable Base64 string represents a harmless IT administration script or a malicious ransomware stager.

Sequential Autoencoders and Large Language Models for Log Anomaly Detection

For high-volume log streams, the SIEM requires a model capable of recognizing when a sequence of events deviates from the established baseline. DeepLog and LogAnomaly represent the foundational architectures for this task⁹. These models utilize Long Short-Term Memory (LSTM) networks to model the sequential nature of log data. During the training phase, the model learns the normal transition probabilities between different log templates¹³. In production, if an incoming sequence of log events violates the predicted pattern, the LSTM flags the sequence¹³.

Recent advancements have introduced Large Language Models (LLMs) to this domain. The

LogLLM architecture utilizes models like LLaMA-3 or BERT to capture deeper semantic relationships within the unstructured text of the logs, outperforming traditional LSTMs by mitigating the impact of out-of-vocabulary terms and parsing errors⁴². The implementation of an LSTM or Transformer-based Autoencoder ensures that contextual anomalies—where individual actions are benign but their specific sequence is malicious—are accurately validated without requiring analysts to mentally stitch together isolated events across multiple dashboards²³.

Further specializing this approach, models such as the wazuh-model (an 8-billion parameter Transformer fine-tuned from LLaMA 3.1 8B by developer kholil-lil) have been explicitly instruction-tuned on labeled SIEM alerts⁴⁵. Operating with 4-bit quantization for rapid inference, this model achieves a 92% accuracy rate in classifying Wazuh alerts as true positives or false positives⁴⁷. Deploying such a model locally ensures high-speed triage while preserving data privacy.

Threat Intelligence Fusion and Dynamic Data Enrichment

While deep learning models excel at analyzing internal structural and behavioral telemetry, they lack visibility into the external reputation of network infrastructure. Therefore, the internal AI verdicts must be aggressively fused with external Threat Intelligence (TI) feeds to achieve definitive alert validation.

To solve the manual data enrichment bottleneck, the architecture must abandon static, slow-updating indicator lists in favor of a dynamic Threat Intelligence Platform (TIP), such as the Malware Information Sharing Platform (MISP). MISP serves as a centralized aggregator, automatically ingesting, deduplicating, normalizing, and scoring data from multiple open-source and commercial feeds¹².

The highest-fidelity open-source feeds required for this integration include:

- **AlienVault Open Threat Exchange (OTX):** Provides massive community-sourced intelligence covering broad malicious IP addresses, domains, and evolving adversary tactics, contributing over 19 million threat indicators daily¹³.
- **Abuse.ch Ecosystem (URLhaus, ThreatFox, SSL Blacklist):** Delivers highly specialized, rapidly updated indicators focused exclusively on malware distribution, botnet command-and-control (C2) servers, and malicious certificates¹⁴.
- **CISA AIS:** Provides authoritative, government-shared indicators utilizing the standard STIX 2.1 and TAXII 2.1 formats¹².

When an alert involving an external IP address is generated, the SIEM does not pause for a human analyst to query VirusTotal. Instead, the orchestration layer instantly queries the local MISP instance via API¹². MISP evaluates the IP against its aggregated feeds, returning a structured confidence score, historical incident correlation, and attribution data³. The integration of these real-time, API-driven lookups into the automated workflow eliminates the necessity for manual swivel-chair verification of network indicators, ensuring that decisions are based on the freshest possible intelligence⁵⁰.

Architectural Implementation: The Wazuh SIEM Deep Learning Bridge

Wazuh provides a highly extensible open-source Extended Detection and Response (XDR) and SIEM foundation. To transition Wazuh from a deterministic rule-based engine into an AI-native platform, the custom ML/DL models must be integrated directly into its alerting pipeline. This is achieved utilizing Wazuh's integratord daemon and the custom-integration architecture⁵².

Configuring the Integratord Daemon

Wazuh allows administrators to execute external Python scripts automatically whenever a specific rule ID, rule group, or severity level is triggered⁵². This mechanism serves as the bridge to the local deep learning inference engine. First, the Wazuh manager's core configuration file (/var/ossec/etc/ossec.conf) must be modified to intercept high-priority alerts and route them to a custom Python script⁵². The configuration block requires defining an integration name prefixed with custom- (e.g., custom-ml-validate), setting the alert format to JSON, and specifying the trigger conditions¹.

```
XML
<ossec_config>
  <integration>
    <name> custom-ml-validate </name>
    <hook_url> http://localhost:8000/v1/analyze </hook_url>
    <level> 10 </level>
    <alert_format> json </alert_format>
  </integration>
</ossec_config>
```

When an alert with a severity level of 10 or higher is generated, the integratord daemon executes the corresponding script located at /var/ossec/integrations/custom-ml-validate¹.

Wazuh Integration Argument	Description	Contextual Use Case
sys.argv[1]	Path to the JSON alert file	Used by the script to open and parse the full alert context.

<code>sys.argv[2]</code>	API Key (from <code>ossec.conf</code>)	Authenticates the request against the external ML inference server.
<code>sys.argv[3]</code>	Hook URL (from <code>ossec.conf</code>)	The destination endpoint for the parsed alert payload.
<code>sys.argv[4]</code>	Path to the full alert file	Provides access to raw, unformatted log data if necessary.

Table: System Arguments passed by Wazuh Integrator to custom Python scripts⁵³.

Python Custom Integration and Local Inference Execution

The Python integration script is engineered to parse the raw JSON payload passed via `sys.argv[1]`, extract the relevant features (such as the command line string, process hierarchy, or network destination), and transmit them to the ML model via the `sys.argv[3]` Hook URL². If the alert involves a PowerShell execution, the Python script calls a local parser to generate the AST of the command string. This structured AST representation is then sent to the locally hosted GNN or Transformer model via an internal API call, utilizing frameworks like FastAPI to expose the PyTorch model endpoints⁵¹.

For straightforward alert triage, specialized fine-tuned LLMs like the previously mentioned `wazuh-model` can be deployed locally using server frameworks like `vLLM` or `llama.cpp`⁴⁵. The Python integration script formats the Wazuh JSON alert into an instruction prompt and submits it to the local LLM. Operating entirely on-premises guarantees that sensitive enterprise telemetry is never transmitted to third-party cloud providers for inference⁴⁵.

Active Response Remediation Protocols

Once the machine learning model processes the data and returns a definitive verdict, the system must act. If the model confirms a high probability of a true positive threat, the integration script proceeds to trigger Wazuh's Active Response module.

Wazuh's Active Response capability allows the system to execute scripts directly on the endpoint (agent) or the manager itself⁵³. An Active Response script (e.g., `custom-ar.py` located in `/var/ossec/active-response/bin/`) is triggered⁵⁹. The Active Response daemon receives the actionable intelligence via standard input (stdin) in JSON format, allowing the script to parse fields such as `srcip` or `process_id`⁵³.

The JSON payload includes a `command` field specifying whether to add (execute the mitigation action) or delete (reverse the action after a specified timeout period)². This enables the SIEM to immediately execute containment protocols, such as appending firewall drop rules or

terminating the malicious process tree identified by the GNN, drastically reducing dwell time.

Automating the Triage Pivot with Shuffle SOAR

While the SIEM handles the deep learning inference and immediate endpoint response, a SOAR platform is strictly required to orchestrate the broader investigation, manage ticketing, and automate the swivel-chair analysis across the enterprise ecosystem. Shuffle, an open-source, containerized SOAR platform designed for high-speed orchestration, is uniquely suited for this architecture⁶⁰.

Ephemeral Container Orchestration

Shuffle differs architecturally from legacy SOAR platforms by leveraging a Docker-centric execution model. It utilizes "Orborus"—a worker node that interfaces directly with the Docker daemon to spawn ephemeral containers for executing workflow actions⁶¹. This design aligns perfectly with a microservices-based security architecture, ensuring environmental consistency and allowing for rapid, parallel execution of complex playbooks⁶³.

The integration begins by creating a workflow in Shuffle equipped with a Webhook trigger⁶⁴. The Wazuh manager is configured to forward ML-validated alerts directly to this Shuffle webhook URI⁶⁴. When Shuffle receives the JSON payload, it initiates the automated verification playbook, acting as the central orchestration nervous system.

Custom App Development and API Execution

To execute the manual pivot steps, Shuffle relies on "Apps," which are API connectors or Python scripts that interface with external platforms⁶². While Shuffle provides a vast library of pre-built integrations, organizations can develop custom apps using the Shuffle Python SDK to accommodate proprietary internal systems⁶⁰.

File Component	Purpose in Shuffle Python SDK
api.yaml	Defines the App's display name, descriptions, HTTP request methods, URL paths, and authentication parameters using OpenAPI standards.
requirements.txt	Specifies the Python dependencies required for the App's execution within its ephemeral Docker container.
app.py	Contains the core Python logic and functions executed by Shuffle when the App is called within a workflow.

Table: Core components of a custom Shuffle App⁶⁶.

Developing a custom app requires configuring three primary files: `api.yaml` to define the OpenAPI specifications, `requirements.txt` for dependencies, and `src/app.py` containing the executable Python functions⁶⁶. For self-hosted Shuffle instances, administrators can utilize the hot-loading feature by placing the app directory into the `shuffle-apps` folder, allowing new integrations to be deployed dynamically without requiring a server restart⁴⁰.

The Automated Verification Playbook

To replicate and automate the exact manual checklist of a Tier 2 analyst, the Shuffle workflow utilizes its visual playbook engine to sequentially query disparate enterprise systems⁴⁰.

First, the workflow parses the username from the Wazuh alert. Shuffle natively executes the Active Directory app, connecting to the domain controller via LDAP to query the user's group memberships, behavioral history, and account status⁶⁴. This instantly determines whether the service account has the authorization to execute the flagged action, eliminating the identity verification bottleneck.

Simultaneously, the workflow extracts any IP addresses, file hashes, or domains from the alert and routes them to the MISP and AlienVault OTX apps⁵¹. Shuffle retrieves the real-time reputation scores and threat attributions, resolving the manual intel verification bottleneck⁷.

For highly specific organizational logic, Shuffle executes raw Python code directly within the workflow, parsing the deep learning verdicts retrieved from Wazuh and formatting the data⁴⁰. Once the automated pivot is complete, Shuffle synthesizes the Active Directory context, the MISP intelligence, and the initial ML validation score from Wazuh into a comprehensive, human-readable narrative⁵⁰. This narrative is automatically pushed to a ticketing system, such as Jira or ServiceNow, via predefined REST API actions, entirely eliminating the administrative burden of manual ticket documentation⁵¹.

By deploying this architecture, the entire multi-dashboard manual pivot is reduced to a sub-30-second automated operation⁵¹. Analysts are completely removed from the initial triage loop and only engage with tickets that have been mathematically validated by the deep learning engine, enriched by threat intelligence, and pre-documented by the orchestration layer.

Appendix: Principal Security Architecture Codebase Redesign

To natively support Graph Neural Networks (GNNs) for process tracking, Abstract Syntax Tree (AST) parsing for deobfuscation, and ephemeral container orchestration for your SOAR engine, the following structural redesign is recommended for your project environment.

Proposed Codebase Tree Structure

.

|-- core-ingest/ (Go: Telemetry & Provenance Graph Pipeline)

```

|-- main.go # Initializes gRPC/HTTP listeners and the data pipeline.
|-- cdm_parser.go # NEW: Translates raw logs (Sysmon/Wazuh) into the Common Data Model (CDM).
|-- correlation.go # Links CDM entities (Processes, Files, Sockets) into Nodes and Edges.
|-- database.go # Stores the graph data in a high-speed database (e.g., Neo4j or Clickhouse).
|-- wazuh_receiver.go # NEW: Dedicated webhook receiver for Wazuh 'integrator' high-severity alerts.
|-- soc-backend/ (Python: AI Inference & SOAR Engine)
|-- main.py # FastAPI server routing alerts from 'core-ingest' to ML agents or playbooks.
|-- ml_inference/ # NEW: AI/ML specific logic
|-- gnn_provenance.py # Runs PyTorch Geometric to analyze the provenance graph for attack paths.
|-- ast_parser.py # Uses 'tree-sitter' to parse PowerShell strings into an Abstract Syntax Tree.
|-- deobfuscator.py # Classifies the AST structural embeddings to detect malicious intent.
|-- llm_triage.py # Local LLM wrapper (e.g., via vLLM) for rapid alert classification.
|
|-- orchestration/ # REFACTORED: The SOAR execution engine
|-- playbooks.py # Defines workflow steps (e.g., query MISP, block IP, isolate host).
|-- sandbox_runner.py # Interfaces with the Docker Daemon to spawn ephemeral execution containers.
|-- active_response.py # Formats containment commands into JSON to trigger Wazuh Active Responses.
|-- context_config.py # Manages API keys for external Threat Intel (AlienVault OTX, Abuse.ch).
-- requirements.txt # Requires: fastapi, docker, torch, torch-geometric, tree-sitter, vllm.
|
|-- deploy/ (Infrastructure & Configuration)
|-- docker-compose.yml # Orchestrates backend, frontend, ingest engine, and message brokers (RabbitMQ).
|-- clickhouse_schema.sql # Configured for fast, temporal graph-edge queries.
|-- wazuh_configs/ # NEW: External SIEM configurations
|-- ossec.conf # Contains the " block pointing Wazuh to your 'wazuh_receiver.go'.
|-- custom-ar.py # Python script deployed to endpoint agents to execute SOAR containment commands.
|-- playbook-sandbox/ # Isolated environment for SOAR actions
|-- Dockerfile.sandbox # Minimal, secure image containing the Python SDK for API interactions.
-- runner.sh # Entrypoint for executing individual playbook steps safely.
-- soc-frontend/ (React: Visual Triage Interface)
-- src/
-- components/
|-- InvestigationGraph.tsx # Renders the GNN-validated attack path as a visual node/edge graph.
-- ThreatIntelPanel.tsx # Displays automated enrichment data retrieved by the SOAR engine.
-- hooks/
`-- useInvestigationStream.ts # Subscribes to real-time WebSockets from the FastAPI backend.

```

Architectural Logic & Module Breakdown

1. The core-ingest Module (The Data Foundation) To run a Graph Neural Network, your data must be mathematically structured as a graph. Your current correlation.go file is supplemented with a cdm_parser.go. This new file is responsible for normalizing heterogeneous telemetry (like Windows Sysmon Event ID 1) into a strict Common Data Model (CDM)³. It translates raw text into a definitive graph structure where entities (users, processes) and their temporal actions (reads, executes, network connections) become mathematically related nodes and edges. The wazuh_receiver.go listens for JSON payloads pushed automatically by Wazuh's integratord daemon⁵².

2. The soc-backend/ml_inference Module (The AI Engine)

This isolates the machine learning models.

- **ast_parser.py and deobfuscator.py:** When an alert containing obfuscated PowerShell arrives, standard text parsing will fail. The ast_parser.py logic uses a library like tree-sitter to convert the raw script into an Abstract Syntax Tree (AST)³⁵. The deobfuscator.py script then passes this structural tree into your machine learning classifier to determine intent, effectively bypassing superficial string scrambling⁷⁰.
- **llm_triage.py:** Deploys a fine-tuned, localized model (such as the wazuh-model) using a high-throughput engine like vLLM to instantly classify alerts as true or false positives without sending sensitive data to cloud providers⁴⁵.

3. The soc-backend/orchestration Module (The SOAR Engine) This redesign shifts your sandbox_runner.py to mimic modern SOAR architectures (like Shuffle), acting as an ephemeral worker node⁶¹. When a playbook is triggered in playbooks.py (e.g., "Enrich IP address"), sandbox_runner.py programmatically spawns an ephemeral Docker container from deploy/playbook-sandbox/Dockerfile.sandbox, executes the external API call, returns the result, and immediately destroys the container. Once a threat is confirmed, active_response.py generates a JSON payload tailored for Wazuh's Active Response mechanism to automatically kill the process on the endpoint⁵³.

4. The soc-frontend Module (The Analyst Interface) Because your backend is now processing data as a mathematical graph, your frontend must visualize it as one. The InvestigationGraph.tsx component consumes the verified subgraphs outputted by gnn_provenance.py²⁸. Instead of reading flat logs, the analyst is presented with a visual attack tree highlighting the exact nodes the GNN identified as malicious.

Conclusion

The reliance on human labor to manually verify overly sensitive security alerts represents a profound architectural failure of modern SOC design. By transitioning to an intelligent, automated pipeline, organizations can systematically dismantle the manual verification bottlenecks that induce alert fatigue and operational paralysis.

Achieving this requires discarding generic models in favor of highly specialized deep learning architectures. By training Graph Neural Networks on DARPA provenance datasets to automate process tree reconstruction, and deploying AST-based Transformers trained on extensive

corpora to deobfuscate malicious code, the SIEM is empowered to make deterministic, high-confidence verdicts.

Integrating these models within an open-source framework—utilizing Wazuh as the telemetry ingestion and local inference bridge, and Shuffle as the containerized orchestration and context-stitching engine—yields a true next-generation architecture. The resulting system does not simply generate alerts; it autonomously investigates, contextualizes, and remediates them. This paradigm shift fundamentally transforms the SOC, restoring human analysts to their proper strategic roles in proactive threat hunting, advanced digital forensics, and sophisticated incident response.

Works cited

1. Revoke-Obfuscation/DataScience/README.md at master - GitHub, <https://github.com/danielbohannon/Revoke-Obfuscation/blob/master/DataScience/README.md>
2. danielbohannon/Revoke-Obfuscation: PowerShell Obfuscation Detection Framework - GitHub, <https://github.com/danielbohannon/revoke-obfuscation>
3. DARPA Transparent Computing Program - Emergent Mind, <https://www.emergentmind.com/topics/darpa-transparent-computing-program>
4. An Event-based Data Model for Granular Information Flow Tracking - USENIX, <https://www.usenix.org/system/files/tapp2020-paper-khoury.pdf>
5. Analyzing the usefulness of the DARPA OpTC dataset in cyber threat detection research - NRC Publications Archive, <https://nrc-publications.canada.ca/fra/voir/auteur/version/?id=258e6cfb-8da9-4251-aaad-002142ccdb4a>
6. Research on APT Detection Model Based on Graph Attention Network and Auto-encoder - Engineering Letters, https://www.engineeringletters.com/issues_v33/issue_7/EL_33_7_35.pdf
7. ICML Poster The Case for Learned Provenance-based System Behavior Baseline, <https://icml.cc/virtual/2025/poster/45219>
8. Flash: A Comprehensive Approach to Intrusion Detection via Provenance Graph Representation Learning - IEEE Computer Society, <https://www.computer.org/csdl/proceedings-article/sp/2024/313000a139/1Ub23WQw20U>
9. Experience Report: Deep Learning-based System Log Analysis for Anomaly Detection - arXiv, <https://arxiv.org/pdf/2107.05908>
10. Log-based Anomaly Detection with Deep Learning: How Far Are We? - arXiv, <https://arxiv.org/pdf/2202.04301>
11. Deep Learning or Classical Machine Learning? An Empirical Study on Log-Based Anomaly Detection - Boxi Yu, <https://boxiyu.github.io/assets/pdf/LightAD.pdf>
12. Threat intelligence tools: 4 categories and 7 buyer criteria - Vectra AI, <https://www.vectra.ai/topics/threat-intelligence-tools>
13. Threat Intelligence - Jigsaw Security, <https://www.jigsawsecurityenterprise.com/threat-intel>
14. Sources | Threat Intelligence Documentation,

- <https://docs.rapid7.com/intelligence-hub/sources/>
15. PowerShell Obfuscation Detection Using Science - Black Hat,
<https://blackhat.com/docs/us-17/thursday/us-17-Bohannon-Revoke-Obfuscation-PowerShell-Obfuscation-Detection-And%20Evasion-Using-Science-wp.pdf>
 16. Detection Lab: Threat hunting and automated detection - DETECTX | Cloud Security Expert, <https://www.detectx.com.au/building-splunk-lab-for-splunking/>
 17. Building Next-Generation Datasets for Provenance-Based Intrusion Detection - NDSS Symposium,
<https://www.ndss-symposium.org/wp-content/uploads/prism2026-21.pdf>
 18. Exploring the Use of Machine Learning to Identify Threats in Powershell Scripts,
https://www.researchgate.net/publication/390619697_Exploring_the_Use_of_Machine_Learning_to_Identify_Threats_in_Powershell_Scripts
 19. Machine Learning Approaches to Malicious PowerShell Scripts Detection and Feature Combination Analysis - Journal of Internet Technology,
<https://jit.ndhu.edu.tw/article/viewFile/3021/3044>
 20. Common Data Model views - IBM,
<https://www.ibm.com/docs/en/networkmanager/4.2.0?topic=dictionary-common-data-model-views>
 21. Analyzing the usefulness of the DARPA transparent computing E5 dataset in APT detection research - ResearchGate,
https://www.researchgate.net/publication/365970387_Analyzing_the_usefulness_of_the_DARPA_transparent_computing_E5_dataset_in_APT_detection_research
 22. ait-aecid/anomaly-detection-log-datasets - GitHub,
<https://github.com/ait-aecid/anomaly-detection-log-datasets>
 23. RNN Anomaly Detection in System Logs | PDF | Parsing | Computing - Scribd,
<https://www.scribd.com/document/983131544/RNN-PROJECT-TOPIC-1>
 24. (PDF) Graph Neural Networks for Intrusion Detection: A Survey - ResearchGate,
https://www.researchgate.net/publication/370733865_Graph_Neural_Networks_for_Intrusion_Detection_A_Survey
 25. Optimizing Network Intrusion Detection Performance with GNN-Based Feature Selection, <https://www.techscience.com/cmc/v85n2/63790/html>
 26. P3GNN: A Privacy-Preserving Provenance Graph-Based Model for APT Detection in Software Defined Networking - arXiv, <https://arxiv.org/html/2406.12003v2>
 27. CONTINUUM: Detecting APT Attacks through Spatial-Temporal Graph Neural Networks, <https://arxiv.org/html/2501.02981v2>
 28. Interpreting GNN-based IDS Detections Using Provenance Graph Structural Features,
https://www.researchgate.net/publication/371222856_Interpreting_GNN-based_IDS_Detections_Using_Provenance_Graph_Structural_Features
 29. Hybrid Time-Position Embedding for Provenance-Based Intrusion Detection - Preprints.org, <https://www.preprints.org/manuscript/202511.1455>
 30. ProvG-Searcher: A Graph Representation Learning Approach for Efficient Provenance Graph Search - arXiv, <https://arxiv.org/html/2309.03647v2>
 31. Learning the APT Kill Chain: Temporal Reasoning over Provenance Data for Attack Stage Estimation - arXiv, <https://arxiv.org/html/2603.07560v1>

32. Provenance Graph Modeling and Feature Enhancement for Power System APT Detection, <https://www.mdpi.com/2079-9292/14/21/4241>
33. AST-Based and Semantics-Preserving Deobfuscation for PowerShell Scripts - GitHub Pages, https://snowroll.github.io/papers/Invoke_Deobfuscation_DSN_2022.pdf
34. ast-grep - GitHub Gist, <https://gist.github.com/eightHundredS/70c9ec82c2b7ba7140dc2cfaa311e8d1>
35. PASTRAL: Privacy-aware AST and TTransformer-based Anomalous command-Line detection - Amazon Science, <https://cdn.amazon.science/0d/30/4db26e484e76a476ccd8ab35a53f/pastral-workshop-camera-ready-v1.pdf>
36. Complete Guide to Threat Intelligence Feeds in 2026 | CyCognito, <https://www.cycognito.com/learn/threat-intelligence/threat-intelligence-feeds/>
37. Detecting PowerShell Obfuscation Techniques using Natural Language Processing - DiVA portal, <https://www.diva-portal.org/smash/get/diva2:1727824/FULLTEXT01.pdf>
38. Static detection of malicious PowerShell based on word embeddings - ResearchGate, https://www.researchgate.net/publication/351398289_Static_detection_of_malicious_PowerShell_based_on_word_embeddings
39. (PDF) Anomaly Detection in Logs Using Deep Learning - ResearchGate, https://www.researchgate.net/publication/386129895_Anomaly_Detection_in_Logs_using_Deep_Learning
40. GitHub - LogIntelligence/LogADEmpirical: Log-based Anomaly Detection with Deep Learning: How Far Are We? (ICSE 2022, Technical Track), <https://github.com/LogIntelligence/LogADEmpirical>
41. Anomaly Detection Using Graph Neural Network - ScholarWorks, <https://scholarworks.boisestate.edu/cgi/viewcontent.cgi?article=3202&context=td>
42. LogLLM: Log-based Anomaly Detection Using Large Language Models - GitHub, <https://github.com/guanwei49/logllm>
43. GANFL: A log anomaly detection method based on collaborative optimization of federated learning and generative adversarial networks - GitHub, <https://raw.githubusercontent.com/mlresearch/v278/main/assets/yao25a/yao25a.pdf>
44. Log-based anomaly detection using BiLSTM-Autoencoder | PDF - Slideshare, <https://www.slideshare.net/slideshow/log-based-anomaly-detection-using-bilstm-autoencoder/283960105>
45. kholil-lil/wazuh-model - Hugging Face, <https://huggingface.co/kholil-lil/wazuh-model>
46. Run Wazuh-model API (Easy Deployment & Flat-Rate Pricing) - Featherless AI, <https://featherless.ai/models/kholil-lil/wazuh-model>
47. Secbert to detect anomalous log entries - Models - Hugging Face Forums, <https://discuss.huggingface.co/t/secbert-to-detect-anomalous-log-entries/176237>
48. Harnessing Deep Learning for Anomaly Detection in Log Data: A Comprehensive

study,

https://www.isecure-journal.com/article_232932_e9cf72601e085c81e2d606c3a2677ec7.pdf

49. Best Threat Intelligence Feeds for Security Teams (2026) - Breachsense, <https://www.breachsense.com/blog/best-threat-intelligence-feeds/>
50. The Best Threat Intelligence Tools & How to Automate Alert Enrichment with Torq, <https://torq.io/blog/threat-intelligence-tools/>
51. Maximus Mukiza | SRE & DevSecOps Engineering, <https://mmt-labs.be/>
52. Integration - Local configuration (ossec.conf) - Wazuh documentation, <https://documentation.wazuh.com/current/user-manual/reference/ossec-conf/integration.html>
53. Wazuh Custom Integrations and Scripting Guide - OpenNix, <https://opennix.org/en/docs/wazuh/development/wazuh-custom-integrations/>
54. How to configure Active Response - Wazuh documentation, <https://documentation.wazuh.com/current/user-manual/capabilities/active-response/how-to-configure.html>
55. How to integrate external software using Integrator - Wazuh, <https://wazuh.com/blog/how-to-integrate-external-software-using-integrator/>
56. GhostPack/Invoke-Evasion: PowerShell Obfuscation and Data Science - GitHub, <https://github.com/GhostPack/Invoke-Evasion>
57. Abdul Rehman abdul4rehman215 - GitHub, <https://github.com/abdul4rehman215>
58. Integrating AI with Wazuh for Advanced Threat Detection - Simplico, <https://simplico.net/2025/02/19/integrating-ai-with-wazuh-for-advanced-threat-detection/>
59. Custom active response scripts - Wazuh documentation, <https://documentation.wazuh.com/current/user-manual/capabilities/active-response/custom-active-response-scripts.html>
60. Shuffle Automation - An Open Source SOAR solution, <https://shuffler.io/>
61. OpenSearch UEBA & SOAR Architecture Guide with Shuffle Integration - Studocu, <https://www.studocu.vn/vn/document/truong-dai-hoc-van-hien/attack-on-titan/opensearch-ueba-soar-architecture-guide-with-shuffle-integration/159360570>
62. Getting Started with Shuffle documentation, https://shuffler.io/docs/getting_started
63. e-Book List, <https://library.iiit.ac.in/e-books%20list%202026.html>
64. Wazuh integration with Shuffle, <https://wazuh.com/blog/wazuh-integration-with-shuffle/>
65. Shuffle extensions documentation, <https://shuffle.empowerists.com/docs/extensions>
66. Shuffle apps documentation, <https://shuffler.io/docs/apps>
67. The Self-Taught Cloud Computing Engineer - KC UMN, <https://kc.umn.ac.id/id/eprint/29665/1/The%20Self-Taught%20Cloud%20Computing%20Engineer.pdf>
68. GitHub - shalin24999/Shuffle-apps: Apps to be used for Shuffle SOAR, <https://github.com/shalin24999/Shuffle-apps>
69. Integrating ServiceNow with Wazuh,

<https://wazuh.com/blog/integrating-servicenow-with-wazuh/>

70. From Obfuscated to Obvious: A Comprehensive JavaScript Deobfuscation Tool for Security Analysis - NDSS Symposium,

<https://www.ndss-symposium.org/wp-content/uploads/2026-f2198-paper.pdf>